结合飞书合同中的应用学习 rxjs

刘文涛 | 1月16日创建

飞书合同代码中使用了不少 rxjs, 在维护时由于对 rxjs 不熟悉导致无法理解原有代码含义, 改动容易出问题.

rxjs 的介绍可以看待凡之前的文档 RxJS 实践 以及官方文档 https://rxjs.dev/guide/overview

本文整理了目前**飞书合同代码中使用到的** rxjs API, 理解了这些 API 在改动代码时就可以理解前人代码含义了, 对飞书合同中未用到的一些 api 的学习也会有帮助.

Obversable

- Observable

Subject

Subject 是多播的 Observable

- Subject
- ReplaySubject
- BehaviorSubject

了解RXJS中四种Subject的区别 | w3cschool笔记

Observable	Subject	BehaviorSubject	ReplaySubject	AsyncSubject
数据生成者	数据生成者和消费者	数据生成者和消费者	数据生成者和消费者	数据生成者和消费者
单播	多播	多播	多播	多播
每个订阅都会产生一个新的实例，每个订阅都是从第一个产生的值开始接收值，所以每个订阅接收到的值都是一样的	将值多播给已经注册监听该 Subject 的观察者们	立即把最新的一个值发送给新的观察者（实例化时需要初始值）	可以存储多个旧值并将它们重播给新的订阅者	把执行 complate 方法后的最后一个值发送给所有订阅者

AsyncSubject 类似于 `last()`，所以大部分情况下不太需要.

```
import { AsyncSubject } from 'rxjs';
const subject = new AsyncSubject();

subject.subscribe({
  next: (v) => console.log(`observerA: ${v}`),
});

subject.next(1);
subject.next(2);
subject.next(3);
subject.next(4);

subject.subscribe({
  next: (v) => console.log(`observerB: ${v}`),
});

subject.next(5);
subject.complete();

// Logs:
// observerA: 5
// observerB: 5
```

Subscription

- Subscription

Scheduler

asyncScheduler

> *Schedule task as if you used setTimeout(task, duration)*

调度器, 下面的 `observeOn` 会讲到

Operators

Creation Operators

from

接收一个参数, 并转化为 observable. A subscription object, a Promise, an Observable-like, an Array, an iterable, or an array-like object to be converted.

> 💡 from 和 of 的区别
>
> - 'of' vs 'from' operator
> - https://www.tektutorialshub.com/angular/rxjs-observable-using-create-of-from-in-angular/
> - of 可以接收任意个参数, 并将参数一个接一个的 emit
> - from 只能接收一个参数, 并且参数需要是可迭代的, 将迭代值 emit 出来

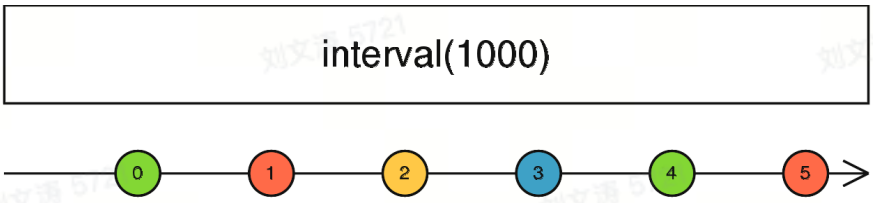
fromEvent

通过事件生成 Observable

- DOM EventTarget
- Node.js EventEmitter
- JQuery-style event target
- DOM NodeList
- DOM HtmlCollection

interval

类似 js 中的 setInterval, 周期性的每隔 x 时间 emit 一次



timer

类似 js 中的 setTimeout, 等待 xx 时间后 emit 一次

```
1 timer(3000)
```

还可以到指定时间才 emit

```
1 timer(new Date(xx
```

或者指定起始时间和间隔时间

```
1 timer(500, 1000);
```

> 💡 `timer(0, 1000)` vs `interval(1000)`
>
> - `timer(0, 1000)` 立即开始, 每 1000ms emit 一次
> - `interval(1000)` 等 1000ms 开始, 每 1000ms emit 一次

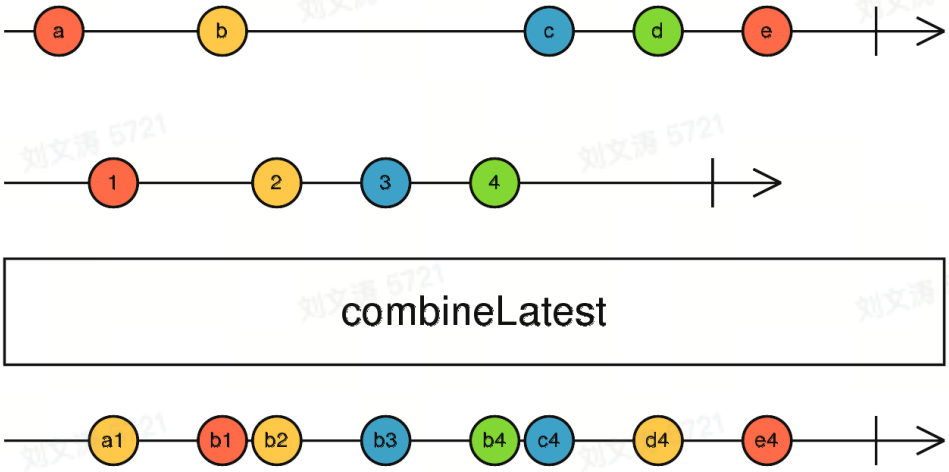
of

可以接收多个参数, 每次 emit 一个值

> 💡 from 和 of 的区别
>
> - 'of' vs 'from' operator
> - https://www.tektutorialshub.com/angular/rxjs-observable-using-create-of-from-in-angular/
> - of 可以接收任意个参数, 并将参数一个接一个的 emit
> - from 只能接收一个参数, 并且参数需要是可迭代的, 将迭代值 emit 出来

Join Creation Operators

combineLatest

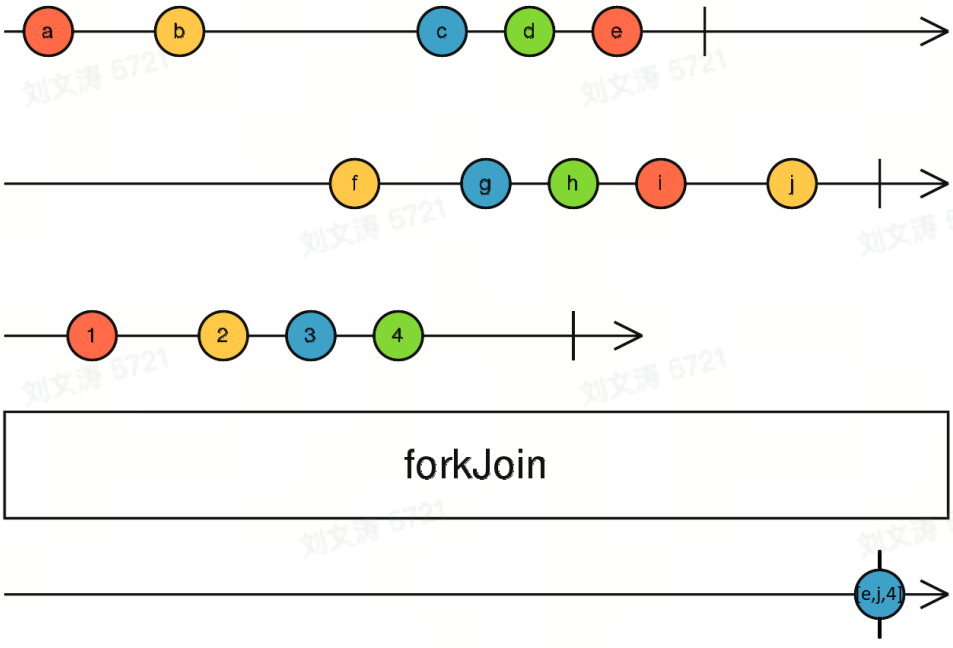


💡 将多个流的最新值组合然后再 emit 出来. 需要等待所有的流都至少有一个值才会启动.

forkJoin

类似 Promise.all, 但是参数既可以是一个数组, 也可以是一个值类型为 `ObservableInput` 的对象

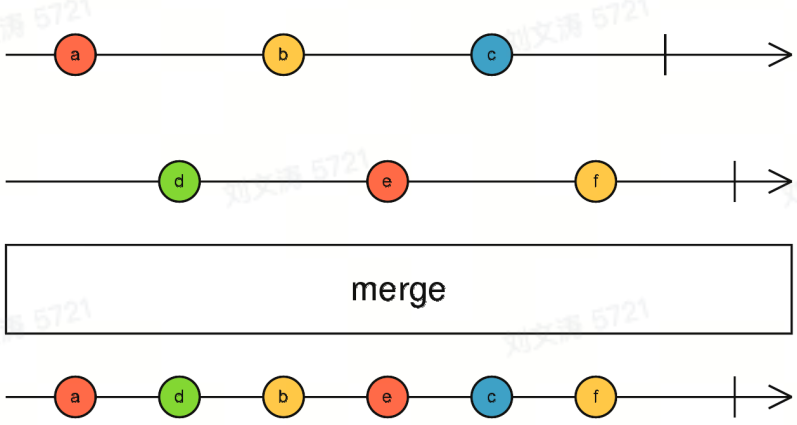
参数: `Array<ObservableInput> | Record<string, ObservableInput>`



💡 需要等所有的流均到达 complete 状态才会触发 emit, 否则会一直等下去.

```
switchMap(() => {
  return forkJoin({
    inData: getAmountTrendAnalysis({
      startDate,
      endDate,
      legalEntityIds,
      tradingPartyIds,
      departmentIds,
      categoryIds,
      aggregationTarget: AggregationTargetType.AMOUNT,
      payTypes: [PayTypeCode.IN],
      emptyBucket: true,
      excludeSourceTypes:
        formatHistoryConditionParams(historyChecked),
    } as any),
    outData: getAmountTrendAnalysis({
      startDate,
      endDate,
      legalEntityIds,
      tradingPartyIds,
      departmentIds,
      categoryIds,
      aggregationTarget: AggregationTargetType.AMOUNT,
      payTypes: [PayTypeCode.OUT],
      emptyBucket: true,
      excludeSourceTypes:
        formatHistoryConditionParams(historyChecked),
    } as any),
  });
}),
```

merge

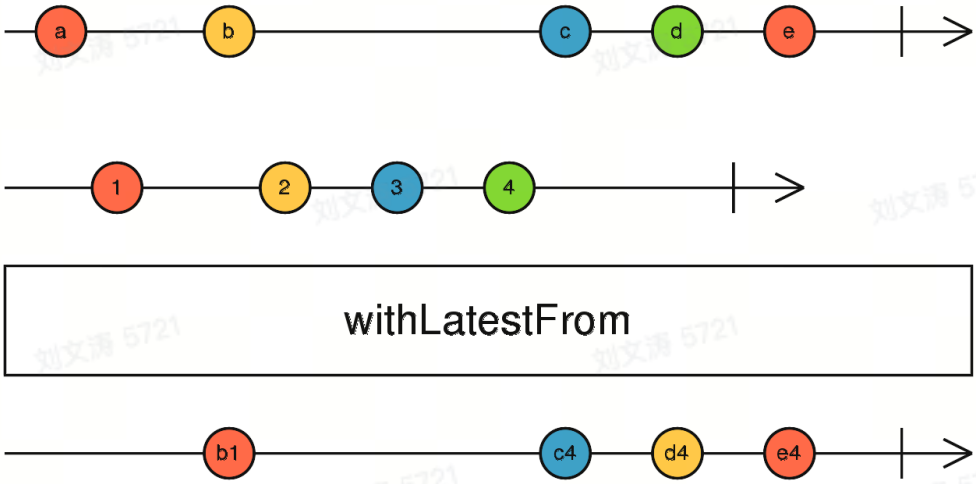


类似于下图的 git merge, 左侧各分支为 merge 前的各 observable, 右侧 git log 为 merge 后的 observable.



Join Operators

withLatestFrom



💡 withLatestFrom 和 combineLatest 的触发方式几乎一样，区别就是withLatestFrom具有主从关系，只有主observable触发时才输出。

类似在 react 中的 useEffect 中获取 useRef 的最新值, 但是 useRef 值的变化并不会触发 useEffect 重新执行.

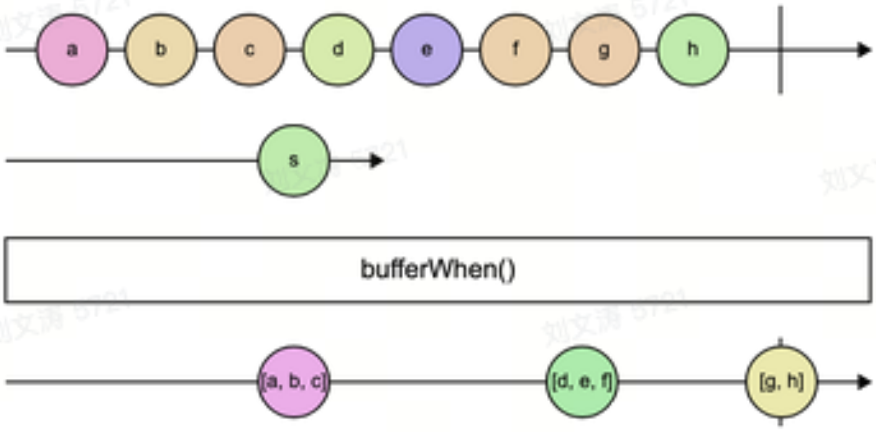
```
// 已保存的表单数据
export const getSavedFormData$ = () =>
  merge(
    contractApplyFormSaved$.pipe(
      // 获取表单的最新值
      withLatestFrom(contractApplyFormChanged$),
      map(([, formData]) => cloneDeep(getPureFormData(formData))),
    ),
    contractApplyFormInit$,
  );
```

Transformation Operators

bufferWhen

收集值，直到关闭选择器发出值才发出缓冲的值。

> 💡 buffer 和 bufferWhen 区别 https://stackoverflow.com/questions/61003061/whats-the-difference-between-the-rxjs-operators-buffer-and-bufferwhen





这里的 bufferWhen 能否用简单的 bufferTime(100) 替代?

```
trigger$.pipe(
  bufferTime(100)
)
```

结论: 即使源数据没有触发值, bufferTime 也会周期性的收集与触发, 而且没有 debounde 效果, 所以用在这里不合适.

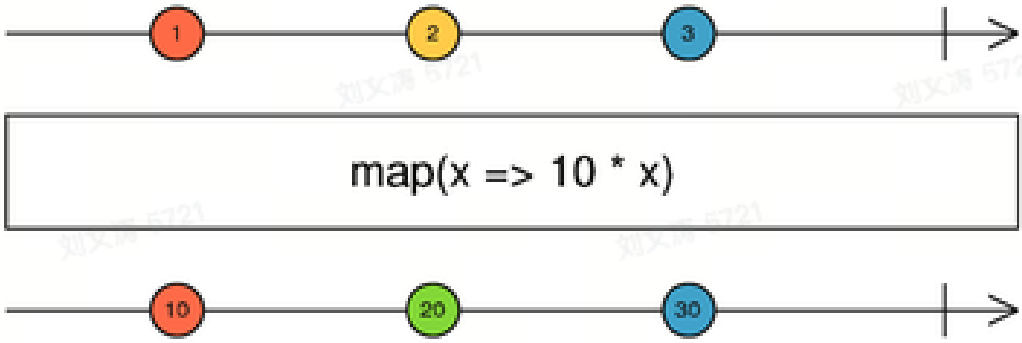
但是可以使用 buffer 稍微简写

```
trigger$.pipe(
  buffer(trigger$.pipe(debounceTime(100)))
)
```

`buffer` subscribes to its argument one time, with each emission from that subscription signalling the current buffer to close and emit, restarting the buffer.

The argument to `bufferWhen` is a function that returns an observable. `bufferWhen` calls it and subscribes to the result immediately, waiting for an emission to signal the close of its buffer. Crucially, the first emission will be the only one that matters, because upon that event, the buffer closes, the subscription to the signal observable ends, and the steps in this paragraph repeat until the observable we're buffering completes.

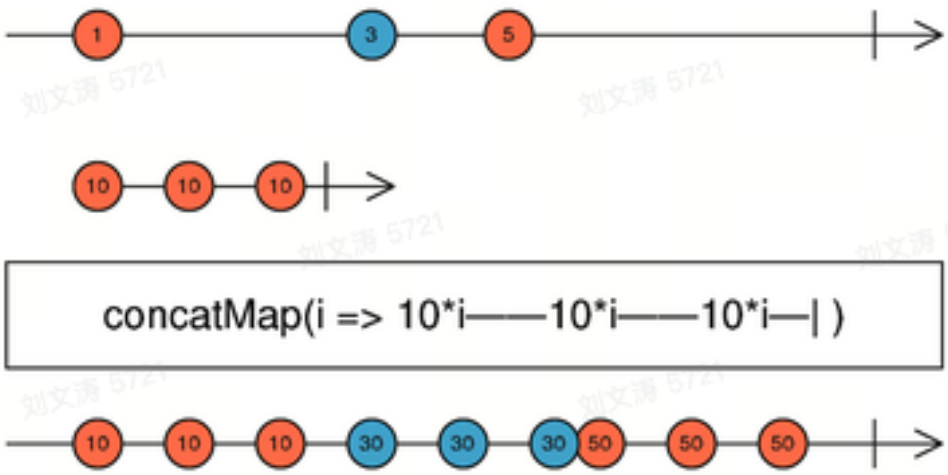
map



concatMap

如下图所示, 输出 1 时, mergeMap 将 1 map 成一个新的 observable, 依此类推, 最终变成 3 个 observable.

可以认为是先 map 再 concat



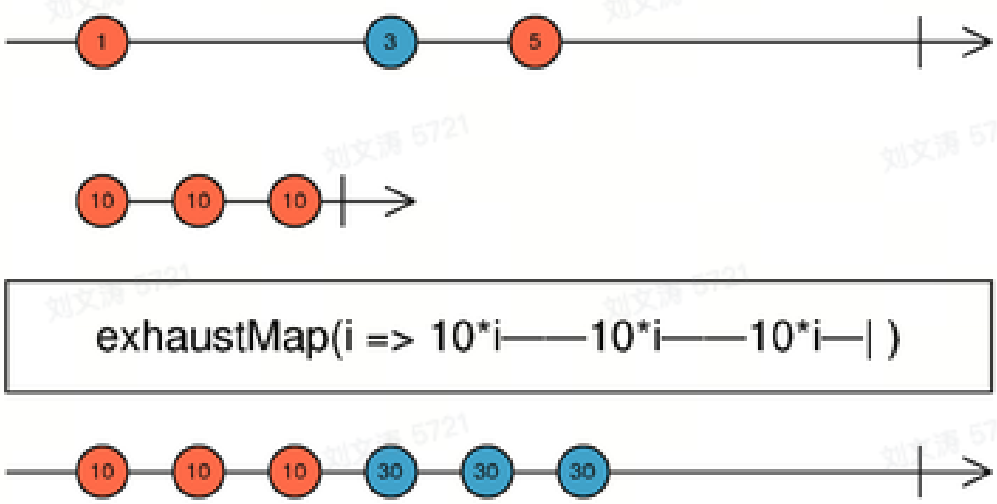
3 和 10 的结合没输出完时, 又输出了5, 此时会等 30 全部输出完后立即输出 50.

这里的 concatMap 是否有更好的写法?

```ts
useObservable(
  () =>
    contractApplySubmitClick$.pipe(
      concatMap(() => {
        return merge(
          contractApplyValidateFail$,
          of(isConfirmProcessError).pipe(filter(e => e)),
        ).pipe(
          take(1),
          concatMap(() => {
            setErrorVisible(true);
            return of(true);
          }),
        );
      }),
      delay(500),
    ),
  () => {
    setPopoverVisible(true);
  },
  [isConfirmProcessError],
);
```

如果 click 连续点击会发生什么?

exhaustMap



源 A 输出的 3 与源 B 的结合还未输出结束, 此时源 A 输出的 5 被阻止(吞掉)了

这里利用 exhaustMap 防止重复请求

```
useSubsObservable(() =>
  reachBottom$.pipe(
    // 设置loading为true
    tap(({ taskTypeName }) => {
      setListRearPending(taskTypeAction(taskTypeName, true));
    }),
    exhaustMap(({ taskTypeName }) =>
      of({}).pipe(
        switchMap(() => getRearList(taskTypeName)),
        takeUntil(search$),
        map(result => {
          if (result.success) {
            // 返回的信息是包含请求的那个taskInstanceId的
            // 所以当请求返回的结果数组长度为1的时候说明请求到头
            // 有列表请求内容的时候
            // 去掉包含最上面一张卡片的列表内容
            const list = result.data?.content || [];
            return list?.length <= 1 ? [] : list;
          }

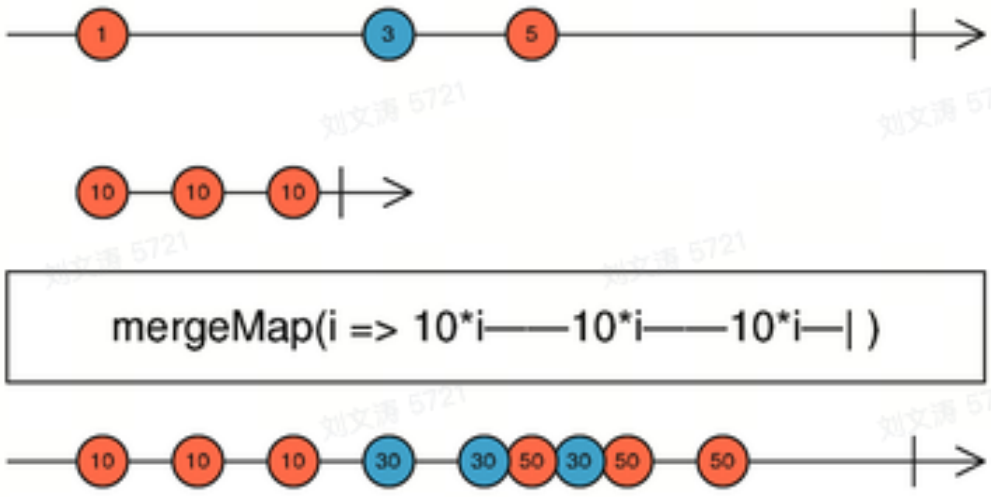
          throw Error(i18n.t('failed_to_get_list', '获取列表失败', {}));
        }),
        tap(list =>
          push(taskTypeAction<TaskQueryItemVO[] | null>(taskTypeName, list)),
        ),
        tap(list => updateLoadedAll(taskTypeName, list)),
        catchError(err => handleError(taskTypeName, err)),
        finalize(() =>
          setListRearPending(taskTypeAction(taskTypeName, false)),
        ),
      ),
    ),
  ),
);
}
```

mergeMap

类似于数学中的多项式相乘.

输出 1 时, mergeMap 将 1 map 成一个新的 observable, 依此类推, 最终变成 3 个 observable.

可以认为是先 map 再 merge

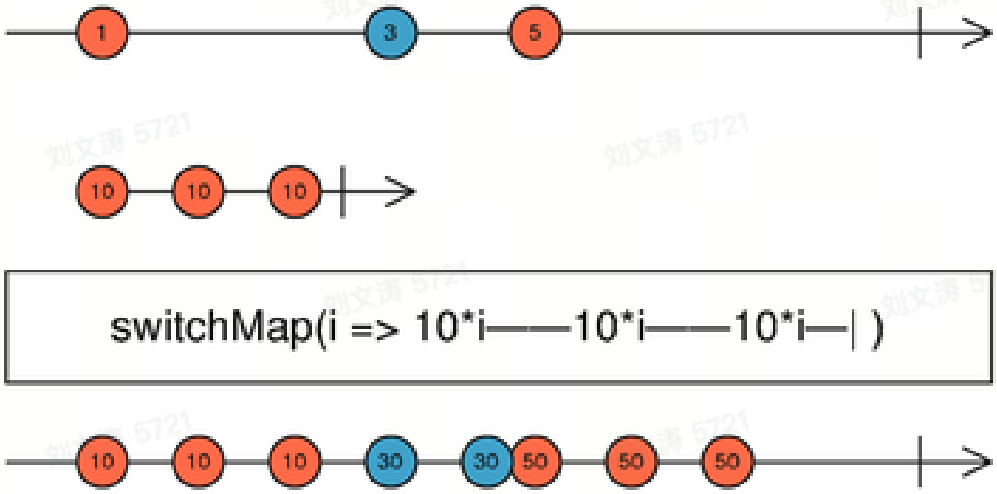


30 还没输出完就开始输出 50, 从10, 30 或 50 看, 各自的时序都是正常的

switchMap

输出 1 时, switchMap 将 1 map 成一个新的 observable, 依此类推, 最终变成 3 个 observable.

每当有新的 observable 到达时, 对于之前 observable 的 subscription 就会被解除.



源 A 输出 5 时, 之前的 3 的结合被阻止了

mergeMap vs switchMap

```
import { of, from } from 'rxjs';
import { mergeMap } from 'rxjs/o
const dummyApi = (character) =>
  return of(`API response for ch
    delay(1000)
  );
}
from(["Calcifer", "Alchemist", "
.pipe(
  mergeMap(arr => dummyApi(arr))
).subscribe( // we subscribe to
)
```

```
import { of, from } from 'rxjs';
import { switchMap, delay } from
const dummyApi = (character) =>
  return of(`API response for ch
    delay(1000)
  );
}
from(["Calcifer", "Alchemist", "
.pipe(
  switchMap(arr => dummyApi(arr)
).subscribe(
  data => console.log(data) // A
)
```

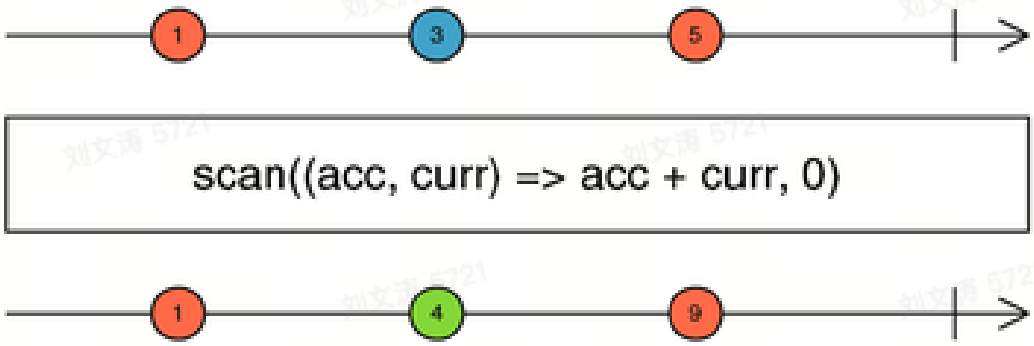
pluck

Deprecated

`pluck('foo', 'bar')` 等同于 `map(value => value?.foo?.bar)`

scan

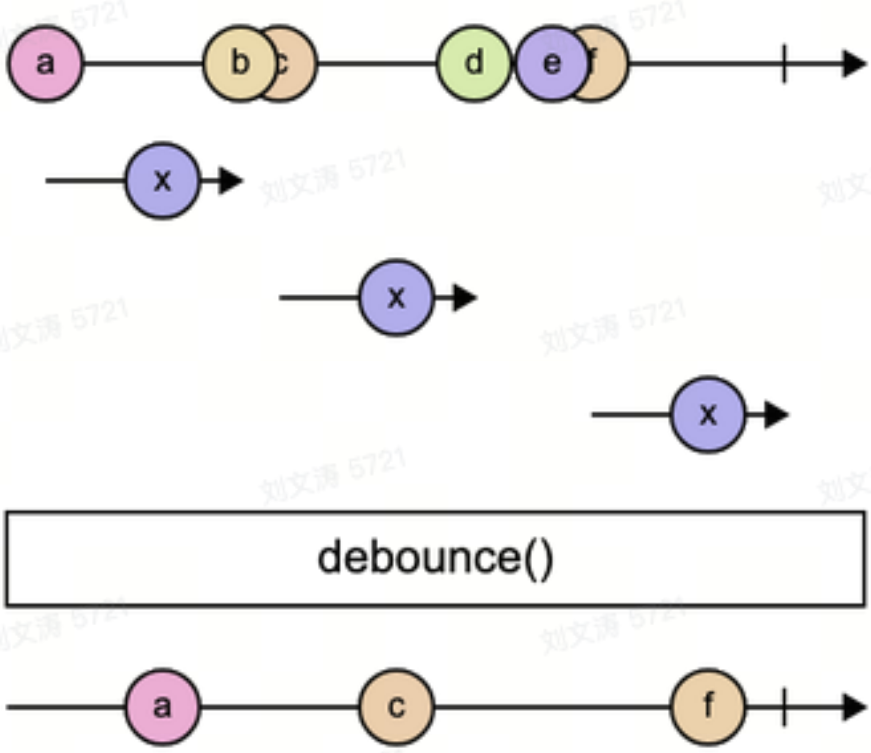
类似 Array 的 reduce 操作



Filtering Operators

debounce

在每次 emit 数据时, 由 debounce 输出的数值决定延时时间




```
import { fromEvent, scan, debounce, interval } from 'rxjs';

const clicks = fromEvent(document, 'click');
const result = clicks.pipe(
  scan(i => ++i, 1),
  debounce(i => interval(200 * i))
);
result.subscribe(x => console.log(x));
```

改写 interval 使用

```
import { fromEvent, scan, debounc

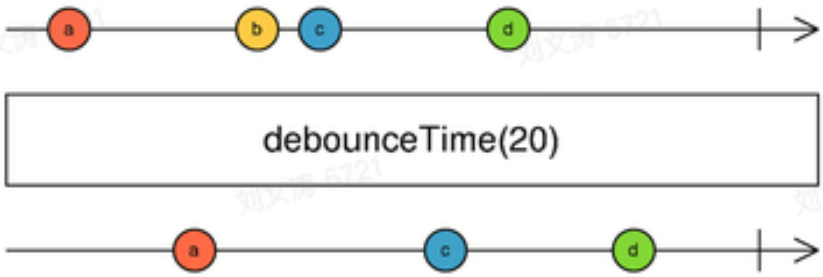
const clicks = fromEvent(documen
const result = clicks.pipe(
  scan(i => ++i, 1),
  debounce(i => timer(200 * i))
);
result.subscribe(x => console.lo
```

```
import { fromEvent, scan, debounc

const clicks = fromEvent(documen
const result = clicks.pipe(
  scan(i => ++i, 1),
  debounce(i => of({}).pipe(dela
);
result.subscribe(x => console.lo
```

debounceTime

上面 debounce 的时间固化版



distinctUntilChanged

过滤数据, 但是可以对前后两个数据进行比较去重

https://rxjs.dev/api/index/function/distinctUntilChanged

```
import { of, distinctUntilChanged } from 'rxjs';

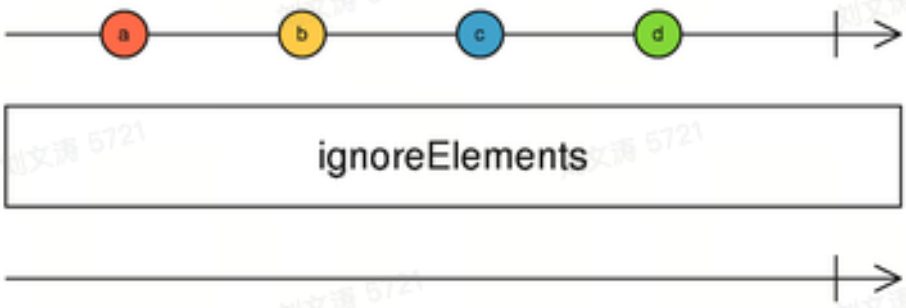
of(1, 1, 1, 2, 2, 2, 1, 1, 3, 3)
  .pipe(distinctUntilChanged())
  .subscribe(console.log);
// Logs: 1, 2, 1, 3
```

filter

和 `Array.prototype.filter` 类似

ignoreElements

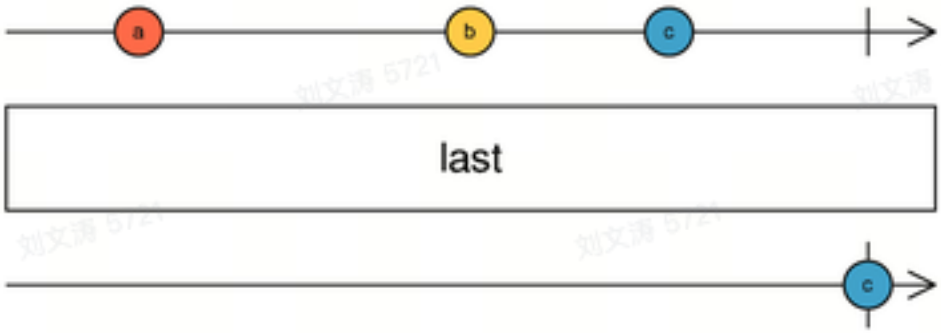
忽略所有输入, 只保留最终的 `error | complete` 事件



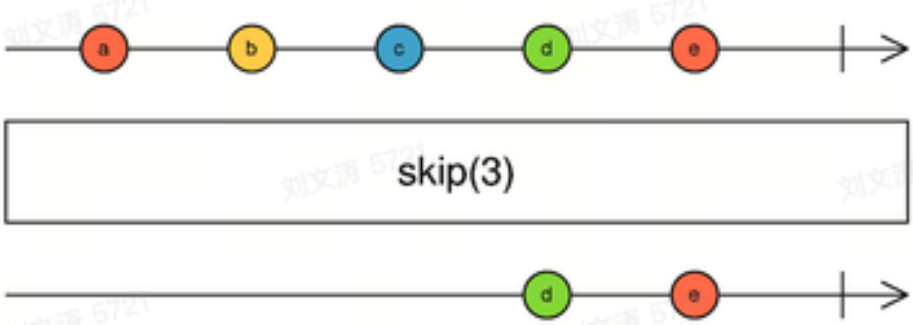

```ts
/**
 * 明细表格调试使用
 */
export const logEpic: RowDataEpic = (input$, { form }) => {
  (window as any)._dynamicForm = form;

  return input$.pipe(
    tap(params => {
      if (['boe', 'pre', 'local'].includes(window.$ea_env)) {
        console.info('rowEpicAll: ', params);
      }
    }),
    ignoreElements(),
  );
};
```

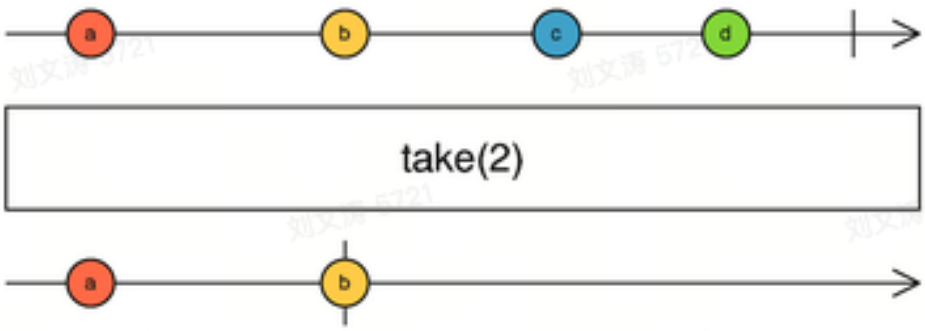
last



skip



take



takeUntil



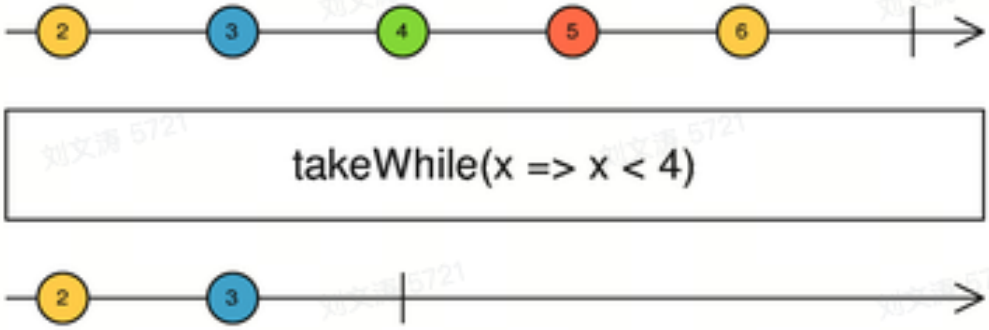
`takeUntil` 是先接收值, 等接收到信号后就不再接收. 如果想反着来, 等接收到信号后再接收值应该怎么办呢?

是否有 `takeWhen` 这样的操作符呢?

答案是 `skipUntil`

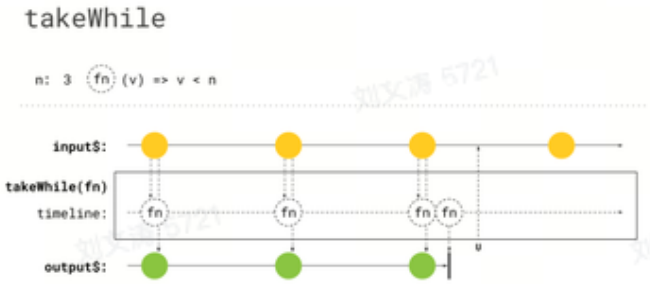


takeWhile



takeUntil 和 takeWhile 区别





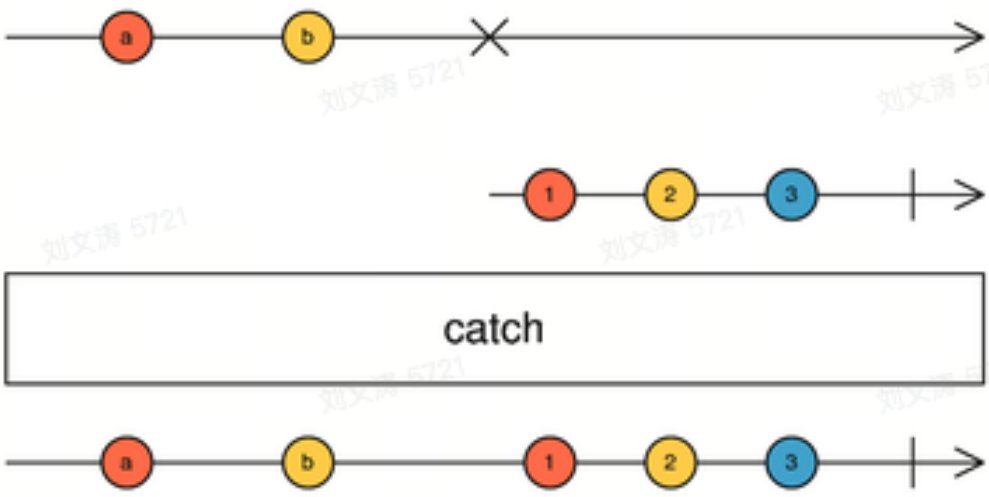
throttle

和 js 中的 dobunce, throttle 概念类似

Multicasting Operators

Error Handling Operators

catchError



```
import { of, map, catchError } f
of(1, 2, 3, 4, 5)
  .pipe(
    map(n => {
      if (n === 4) {
        throw 'four!';
      }
      return n;
    }),
    catchError(err => of('I', 'I
  )
  .subscribe(x => console.log(x)
  // 1, 2, 3, I, II, III, IV, V
```

```
import { of, map, catchError, ta
of(1, 2, 3, 4, 5)
  .pipe(
    map(n => {
      if (n === 4) {
        throw 'four!';
      }
      return n;
    }),
    catchError((err, caught) =>
    take(30)
  )
  .subscribe(x => console.log(x)
  // 1, 2, 3, 1, 2, 3, ...
```

Utility Operators

tap

副作用方法, 每个值都会触发, 但是这个方法的结果并不会对流的后续走向造成影响.

observeOn

> Re-emits all notifications from source Observable with specified scheduler.

在源和最终的观察者之间引入了一个代理观察者, 可以控制代理观察都何时发出值.

https://www.w3cschool.cn/rxjs/rxjs-htv63cix.html

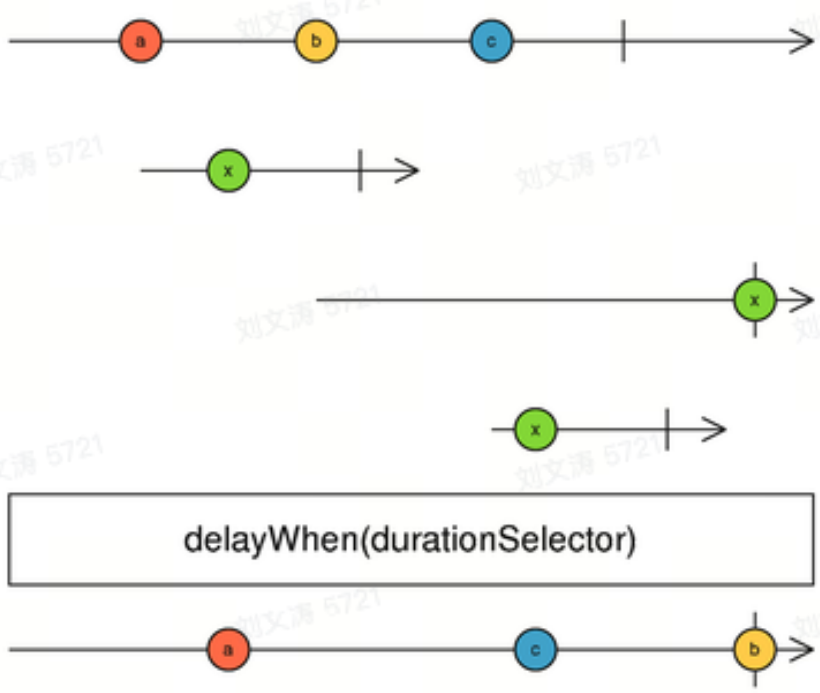
`of(1,2,3, asyncScheduler)` 和 `of(1,2,3).pipe(observeOn(asyncScheduler))` 区别可以看下面这篇文章, 介绍的比较详细

https://medium.com/angular-in-depth/rxjs-applying-asyncscheduler-as-an-argument-vs-with-observeon-operator-2340af3b6cac

delayWhen

和 delay 功能类似, 但是 delay 时长由第 2 个 obversable 决定.

和 bufferWhen 中第 2 个 observable 用法一样.



Conditional and Boolean Operators

Mathematical and Aggregate Operators

mergeWith

和 merge 功能一样. 区别在于 mergeWith 用在 pipe 中, 用来和主 observable 合并. merge 直接合并两个 observable 并返回一个新的 observable

```
import { fromEvent, map, mergeWi
const clicks$ = fromEvent(docume
const mousemoves$ = fromEvent(do
const dblclicks$ = fromEvent(doc
mousemoves$
  .pipe(mergeWith(clicks$, dblcl
  .subscribe(x => console.log(x)
```

```
import { merge, fromEvent, inter
const clicks = fromEvent(documen
const timer = interval(1000);
const clicksOrTimer = merge(clic
clicksOrTimer.subscribe(x => con
```

firstValueFrom

取第一个值, 返回一个 Promise

> 💡 对应的有 lastValueFrom, 但是只有在源触发 complete 后才会触发.

combineLatestWith

combineLatest 的 pipe 版

zipWith

zip 的 pipe 版

```
of(27, 25, 29).pipe(
  zipWith(
    of('Foo', 'Bar', 'Beer'),
    of(true, true, false)
  )
).subscribe(x => console.log(x))
// [27, 'Foo', true]
// [25, 'Bar', true]
// [29, 'Beer', false]
```

```
// deprecated
// Replaced with zipWith.
// Will be removed in v8.
zip(
  of(27, 25, 29)
  of('Foo', 'Bar', 'Beer'),
  of(true, true, false)
).subscribe(x => console.log(x))
// [27, 'Foo', true]
// [25, 'Bar', true]
// [29, 'Beer', false]
```

pipe

> pipe() can be called on one or more functions, each of which can take one argument ("UnaryFunction") and uses it to return a value. It returns a function that takes one argument, passes it to the first UnaryFunction, and then passes the result to the next one, passes that result to the next one, and so on.

这里的 pipe 不是在 Observable 实例上调用的 pipe 方法, 但是用法类似.

```
of(1, 2, 3).pipe(
  map(num => num * 10),
  map(num => num - 5),
  filter(num => num < 20)
).subscribe(value => {
  console.log(value);
});
```

```
function reusablePipeSet() {
  return pipe(
    map((num: number) => num * 1
    map(num => num - 5),
    filter(num => num < 20)
  );
}

of(1, 2, 3).pipe(
  reusablePipeSet()
).subscribe(value => {
  console.log(value)
});
```

参考文档: https://writtenforcoders.com/blog/RxJS-Pipe-the-complete-guide

finalize

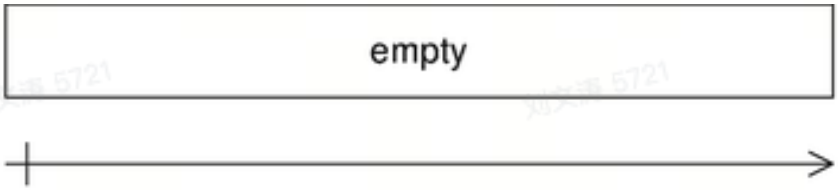
和 tap 功能类似, 但是只在 complete 事件时触发.

可以传入 side effect, `ignoreElements()` 不支持传入 side effect

ObservableInput

EMPTY

类似于经常用到的 noop 方法, 不做任何操作, 占位




```ts
import { interval, mergeMap, of, EMPTY } from 'rxjs';

const interval$ = interval(1000);
const result = interval$.pipe(
  mergeMap(x => x % 2 === 1 ? of('a', 'b', 'c') : EMPTY),
);
result.subscribe(x => console.log(x));

// Results in the following to the console:
// x is equal to the count on the interval, e.g. (0, 1, 2, 3, ...)
// x will occur every 1000ms
// if x % 2 is equal to 1, print a, b, c (each on its own)
// if x % 2 is not equal to 1, nothing will be output
```



```ts
// 获取要更新的信息
const subscription = getSavedFormData$()
  .pipe(
    observeOn(asyncScheduler),
    distinctUntilChanged(isDepsInContractChanged),
    concatMap(() => {
      const currentValues = valueRef.current;
      const currentContract = getContractFormData?.();

      // 没有数据，直接结束不产生联动
      if (!currentValues?.length) {
        return EMPTY;
      }

      // 积累的行数据变化
      let acmPatchRowValues: Record<string, ObjectType> = {};

      epicStore.trigger({
        newValues: currentValues || [],
        contract: currentContract,
        forceTrigger: true,
      });
```

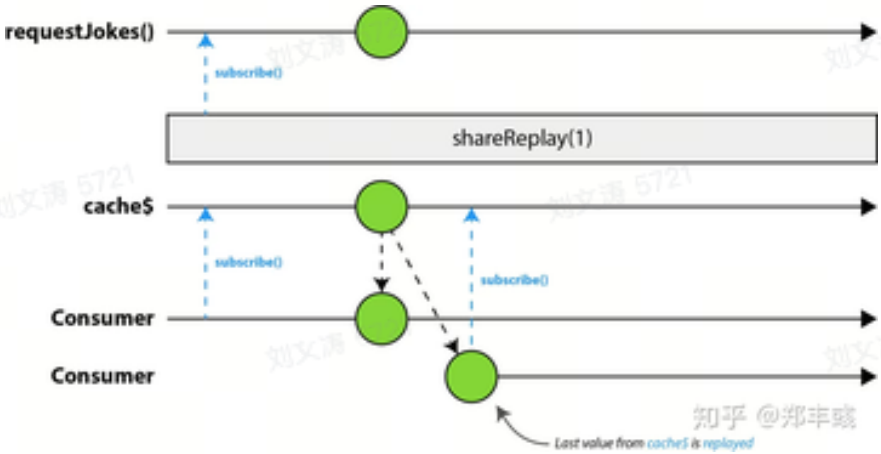
返回 EMPTY 后就不会继续走下面的 pipe 流程

EMPTY 内部实现, 一个立即 complete 的 observable

```
1 export const EMPTY = new Observable<never>((subscriber) => subscriber.complet
```

shareReplay

共享源 observable 并重放指定次数的发出



相关链接 https://zhuanlan.zhihu.com/p/42264563

```
// Epic上下文存储
this.epicMapper$ = value$.pipe(
  // 初始值
  mergeWith(of(valueRef.current || [])),
  // epicMapper状态管理
  scan((epicMapper, values) => {
    const newValueIdSet = new Set(
      (values || []).map(_row => get(_row, rowKey)),
    );
    const currValueRowIdSet = new Set(epicMapper.keys());

    Array.from(newValueIdSet).forEach(_id => {
      if (currValueRowIdSet.has(_id)) {
        currValueRowIdSet.delete(_id);
      } else {
        // 新增id
        epicMapper.set(_id, new EpicRowState({ epic, ctx }));
        // console.info('debugger create epic state', groupKey, _id);
      }
    });

    // 删除的id
    Array.from(currValueRowIdSet).forEach(_id => {
      epicMapper.delete(_id);
      // console.info('debugger delete epic state', groupKey, _id);
    });

    return epicMapper;
  }, new Map<string, EpicRowState>()),
  // 缓存当前epic
  shareReplay(1),
);
```

如果不加 shareReplay, 则获取不到缓存的值, 只能等到 value$ 触发最新的值后才能拿到.

> 是 ReplaySubject 的一个变种, 内部就是用 ReplaySubject 实现的

> 这里并不适合用 buffer(1)
>
> buffer(1) 是满 1 个数据会输出, 输出后就没有值了. 新的 subscrption 并不会拿到最新的值.